# Vaidik Dave

Applied AI / LLM Engineer

India · Open to remote

davevaidik20@gmail.com | +91 80947 29441 | github.com/Vaidik-Dave23 | linkedin.com/in/vaidik-dave-0457873ab | vaidik-dave23.github.io

## Summary

Applied AI engineer who ships production LLM systems end to end as a solo developer: RAG pipelines, LLM-as-judge evaluation, LangGraph agents, and fault-tolerant model failover. Two live products on Azure and Vercel and a retrieval benchmark, each backed by its own evaluation or observability layer.

## Projects

**Auvon.ai** — AI study workspace — Live · Code

FastAPI · React · MongoDB Atlas · Gemini · OpenAI · Azure App Service · Vercel

- Built a **cascading LLM failover** across multiple Gemini keys and models (3.5 Flash → 2.5 Flash) with GPT-4o-mini as the final fallback; per-key daily-quota blacklisting keeps generation up when free-tier keys run out.
- Built PDF RAG chat on a custom markdown-aware chunker, Gemini embeddings and per-user cosine retrieval with a 0.70 similarity cutoff.
- Wrote an **LLM-as-judge harness** (Gemini 2.5 Flash judge) that benchmarks the RAG pipeline end to end: **81% faithfulness, 99% relevance** on 19 auto-generated Q&A across 4 topics; traced every faithfulness miss to the model adding outside knowledge, not retrieval errors.
- Instrumented the API with structured JSON logs, request-ID middleware, **Prometheus** metrics for LLM latency and tokens per provider/model, and **OpenTelemetry** traces, viewed in Grafana and Jaeger.
- Shipped AI study roadmaps, notes from topics, PDFs and YouTube videos, and quizzes with weak-topic review; Google OAuth + JWT auth, per-user rate limiting, CI/CD to Azure via GitHub Actions.

**Recoup** — AI payment-recovery platform — Live · Code

LangGraph · Gemini · FastAPI · PostgreSQL · Razorpay · React/TypeScript · Azure · Vercel

- Built a **LangGraph recovery workflow** (triage → strategy → content → execute or escalate) that turns failed payments into Razorpay test-mode payment links sent over SMTP email.
- Put a **deterministic policy layer** between the model and every action: hard declines always go to a human with an AI-written handoff, and unsupported channels are overridden in code, not by prompt.
- Added **idempotency guards** against duplicate payment links and double sends, logged every agent step to Postgres as an audit trail, and scoped all data per merchant behind JWT auth.

**DocMind** — multi-document RAG with a retrieval benchmark — Code

FastAPI · FAISS · Sentence-Transformers · Gemini

- Built multi-document RAG over a **502-page legal corpus** with FAISS and all-MiniLM-L6-v2 embeddings.
- Cut evaluation cost from **8 to 2 LLM calls per query** with a custom RAGAS-style harness.
- Benchmarked fixed-size vs semantic chunking on Recall@k and MRR; found the winner flips with how gold labels are defined, and traced semantic-chunk misses to 19 of 83 chunks exceeding MiniLM's 256-token limit.

## Skills

| | |
|---|---|
| **LLM & AI** | RAG, embeddings, chunking strategies, LLM-as-judge and RAGAS-style evaluation, LangGraph agents, LangChain, prompt design, Gemini and OpenAI APIs, FAISS, Sentence-Transformers |
| **Backend** | Python, FastAPI, Pydantic, REST APIs, JWT and Google OAuth, rate limiting, Redis |
| **Data** | MongoDB, PostgreSQL, SQLite, Pandas, NumPy |
| **Infra & Ops** | Docker, Azure App Service, Google Cloud Run, Vercel, GitHub Actions, Prometheus, Grafana, OpenTelemetry |
| **Frontend** | React, TypeScript, Vite, Tailwind CSS |

## Education

**BS in Data Science and Applications** · Indian Institute of Technology Madras (online degree) — Expected May 2027

**Bachelor of Computer Applications** · Jai Narain Vyas University, Jodhpur — 2026